Basilio Andrés T11

# Tipos de datos en PostgreSQL

El tipo de dato de una columna define qué valores acepta, cuánto espacio ocupan y qué operaciones se pueden hacer con ellos. Elegir bien el tipo ayuda a validar la información y evita desperdiciar espacio.

## Numéricos

| Tipo | Tamaño | Formato | Capacidad |
|---|---|---|---|
| smallint | 2 bytes | 150 | $-32\,768$ a $32\,767$ |
| integer | 4 bytes | 321137 | $-2\,147\,483\,648$ a $2\,147\,483\,647$ |
| bigint | 8 bytes | 9007199254 | $-9.22 \times 10^{18}$ a $9.22 \times 10^{18}$ aprox. |
| numeric(p,s) | variable | 1520.75 | Hasta $131\,072$ dígitos antes del punto decimal y $16\,383$ después; es exacto |
| real | 4 bytes | 3.1416 | Punto flotante con 6 dígitos de precisión; es aproximado |
| double precision | 8 bytes | 2.718281828 | Punto flotante con 15 dígitos de precisión; es aproximado |
| smallserial | 2 bytes | automático | 1 a $32\,767$ |
| serial | 4 bytes | automático | 1 a $2\,147\,483\,647$ |
| bigserial | 8 bytes | automático | 1 a $9.22 \times 10^{18}$ aprox. |

En numeric(p,s), $p$ es el total de dígitos y $s$ los que van después del punto, de modo que numeric(7,2) guarda hasta 99999.99. Para dinero conviene numeric, porque real y double precision pueden acumular errores de redondeo. Los tipos serial no son tipos reales: crean un entero con una secuencia que genera el siguiente valor.

```
create table producto(
  id_producto serial primary key,
  existencia smallint not null,
  precio numeric(7,2) not null
);
insert into producto(existencia, precio) values (25, 149.90);
```

## Caracter

| Tipo | Descripción y capacidad |
|---|---|
| char(n) | Longitud fija de $n$ caracteres. Si el texto es más corto, se completa con espacios |
| varchar(n) | Longitud variable con un máximo de $n$ caracteres. Si el texto es más largo, marca error |
| text | Longitud variable sin un límite declarado. Un valor puede ocupar hasta 1 GB aproximadamente |

Los valores se escriben entre comillas simples. char(n) sirve para datos de tamaño fijo como la CURP, varchar(n) para datos con un máximo razonable como un nombre, y text para comentarios largos. En PostgreSQL los tres tienen un rendimiento prácticamente igual.

```
create table alumno(
  curp char(18) primary key,
  nombre varchar(100) not null,
  comentarios text
);
insert into alumno values
  ('LOPA010101MDFPNN09', 'Ana Lopez', 'Alumna de nuevo ingreso');
```

## Fecha y hora

| Tipo | Tamaño | Formato | Capacidad |
|---|---|---|---|
| date | 4 bytes | '2026-11-02' | Del 4713 a. C. al 5 874 897 d. C. con resolución de un día |
| time | 8 bytes | '14:30:00' | De 00:00:00 a 24:00:00 con resolución de un microsegundo |
| timestamp | 8 bytes | '2026-11-02 14:30:00' | Del 4713 a. C. al 294 276 d. C. con resolución de un microsegundo |
| timestamptz | 8 bytes | '2026-11-02 14:30:00-06' | Mismo rango que timestamp, considerando zona horaria |
| interval | 16 bytes | '2 days 3 hours' | Hasta ±178 000 000 años |

El formato recomendado es el ISO 8601, año-mes-día, porque no se presta a confusiones. Restar dos fechas da un número de días y sumar un interval a una fecha da otra fecha.

```
select date '2026-11-28' - date '2026-11-02';  -- 26
select date '2026-11-02' + interval '1 month';
```

## Otros tipos

### boolean

Ocupa 1 byte y guarda verdadero, falso o nulo. Se escribe true o false. Es útil para banderas, como indicar si una persona es titular de una cuenta.

```
create table cuenta(
  num_cuenta integer primary key,
  es_titular boolean not null default false
);
```

### uuid

Ocupa 16 bytes y guarda un identificador de 128 bits, escrito como 32 dígitos hexadecimales separados por guiones. Sirve para generar identificadores sin que varios servidores tengan que coordinarse. Desde la versión 13, gen_random_uuid() genera uno.

```
create table sesion(
  id_sesion uuid primary key default gen_random_uuid()
);
-- a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11
```

### jsonb

Guarda documentos JSON en un formato binario, con un tamaño variable. A diferencia de json, que conserva el texto tal como llegó, jsonb permite consultas más rápidas y crear índices sobre su contenido. Sirve cuando los datos no tienen siempre los mismos campos.

```
create table articulo(
  id_articulo serial primary key,
  datos jsonb
);
insert into articulo(datos)
  values ('{"marca": "Norma", "hojas": 100}');
select datos ->> 'marca' from articulo;  -- Norma
```

Me llamaron la atención porque boolean evita validar a mano valores como S o N, uuid resuelve la generación de identificadores en sistemas distribuidos, y jsonb permite combinar datos relacionales con datos flexibles sin salir de PostgreSQL.

## Referencia


- The PostgreSQL Global Development Group. *PostgreSQL Documentation*, capítulo 8: Data Types. postgresql.org/docs/current/datatype.html